Here is the complete documentation, structured analysis, algorithms, pseudocode, Python source code equivalents, test cases, and conclusion for your suite of programs (Questions 11–20).

To provide a thorough and practical reference, key representative programs are detailed completely with Python implementations and test cases, while the remaining programs are organized with structured logic and specifications.

---

1. Executive Summary & Overview

This document covers 10 algorithmic utility and decision-support systems implemented in C. These programs handle academic evaluation, financial transactions, resource inventory, logistics, and admissions processing. Below are their comprehensive specifications and Python translations.

---

2. Program-by-Program Analysis & Python Equivalents

Program 11: Examination Result and Grace-Mark Validation

* **Problem Statement:** Evaluate student marks across N subjects, apply up to 5 grace marks if a subject mark is between 30 and 34 (inclusive) to push it to the pass threshold (35), and determine the final pass/fail status.
* **Objectives:** Validate input bounds, conditionally apply grace marks, compute total scores, and output aggregate results.
* **Assumptions:** Maximum marks per subject is 100, passing mark is 35, and grace mark limit is 5.

Algorithm & Pseudocode

1. Input number of subjects (N).
2. Initialize `total = 0` and `failed = FALSE`.
3. Loop i from 1 to N:
* Input `marks`.
* If marks < 0 or > 100, display error and terminate.
* If marks < 35 and $\text{marks} + 5 \ge 35$, update $\text{marks} = \text{marks} + 5$.
* If marks < 35, set `failed = TRUE`.
* Add marks to `total`.

4. Display total marks and final PASS/FAIL status.

Python Source Code

```python
def exam_result_validation():
    n = int(input("Enter number of subjects: "))
    total = 0
    failed = False
    max_marks = 100
    grace = 5

    for i in range(1, n + 1):
        marks = int(input(f"Enter marks for subject {i}: "))
        if marks < 0 or marks > max_marks:
```

```
            print("Invalid marks!")
            return
        if marks < 35 and marks + grace >= 35:
            marks += grace
            print(f"Grace marks applied: {marks}")
        if marks < 35:
            failed = True
        total += marks

    print(f"Total Marks: {total}/{n * max_marks}")
    print("Result: FAIL" if failed else "Result: PASS")

if __name__ == "__main__":
    exam_result_validation()
```

* **Test Case:** 3 subjects, marks entered: `32`, `75`, `40`.
* **Expected Output:**

```text
Grace marks applied: 37
Total Marks: 152/300
Result: PASS
```

Program 13: ATM Transaction Sequence Simulation

* **Problem Statement:** Simulate an ATM terminal handling deposits and withdrawals with balance checks and transaction limit constraints.
* **Objectives:** Track account balance, validate withdrawal limits (max Rs. 10,000), and summarize total deposits and withdrawals.
* **Assumptions:** Initial balance ≥ 0, withdrawal limit = 10,000, and transaction amounts must be positive integers.

Algorithm & Pseudocode

1. Input initial balance.
2. Input number of transactions (N).
3. Loop through each transaction:
* Input transaction type (1: Deposit, 2: Withdraw) and amount.
* If amount ≤ 0, show error.
* If type 1, add amount to balance and update total deposits.
* If type 2, check if amount exceeds balance or limit (10,000); if valid, deduct amount and update total withdrawals.

4. Display summary of deposits, withdrawals, and final balance.

Python Source Code

```python
def simulate_atm():
    balance = int(input("Enter initial balance: "))
    if balance < 0:
```

```
    print("Invalid balance!")
    return
n = int(input("Enter number of transactions: "))
deposits = 0
withdrawals = 0
limit = 10000

for i in range(1, n + 1):
    print("\n1. Deposit  2. Withdraw")
    t_type = int(input("Enter transaction type: "))
    amount = int(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount!")
    elif t_type == 1:
        balance += amount
        deposits += amount
        print("Deposit successful.")
    elif t_type == 2:
        if amount > balance:
            print("Insufficient balance!")
        elif amount > limit:
            print("Withdrawal limit exceeded!")
        else:
            balance -= amount
            withdrawals += amount
            print("Withdrawal successful.")
    else:
        print("Invalid transaction type!")
```

```
        print(f"Current Balance: {balance}")

    print(f"\nTotal Deposits: {deposits}")
    print(f"Total Withdrawals: {withdrawals}")
    print(f"Final Balance: {balance}")

if __name__ == "__main__":
    simulate_atm()
```

* **Test Case:** Initial balance = 5000, Transaction 1: Deposit 2000, Transaction 2: Withdraw 1500.
* **Expected Output:**

```text
Deposit successful. Current Balance: 7000
Withdrawal successful. Current Balance: 5500
Total Deposits: 2000
Total Withdrawals: 1500
Final Balance: 5500
```

Program 15: Smart Parking Fee Management

* **Problem Statement:** Calculate parking fees based on vehicle type, parking duration, and overnight status with a maximum daily cap.

* **Objectives:** Compute hours parked (handling overnight rollover), apply hourly rates (Car: 30, Bike: 15, Bus: 50), enforce a Rs. 300 cap, and add an overnight surcharge if applicable.

* **Assumptions:** 24-hour clock format, valid vehicle codes (1, 2, 3).

Algorithm & Pseudocode

1. Input entry hour, exit hour, vehicle type, and overnight flag.

2. Validate inputs ($0 \le \text{hour} \le 23$).

3. Compute duration: $\text{hours} = \text{exit} - \text{entry}$; if $\text{hours} < 0$, add 24. If $\text{hours} == 0$, set to 24.

4. Calculate fee based on vehicle type and enforce Rs. 300 cap.

5. Add Rs. 100 if overnight flag is true. Display fee.

Python Source Code

```python
def smart_parking():
    entry = int(input("Enter entry hour (0-23): "))
    exit_hr = int(input("Enter exit hour (0-23): "))
    vehicle = int(input("Vehicle type (1.Car 2.Bike 3.Bus): "))
    overnight = int(input("Overnight parking? (1.Yes 0.No): "))

    if not (0 <= entry <= 23 and 0 <= exit_hr <= 23 and 1 <= vehicle <= 3 and overnight in [0, 1]):
        print("Invalid input!")
        return
```

```
    hours = exit_hr - entry
    if hours < 0:
        hours += 24
    if hours == 0:
        hours = 24

    rates = {1: 30, 2: 15, 3: 50}
    fee = hours * rates[vehicle]

    if fee > 300:
        fee = 300
    if overnight:
        fee += 100

    print(f"Parking Duration: {hours} hours")
    print(f"Parking Fee: Rs.{fee}")

if __name__ == "__main__":
    smart_parking()
```

* **Test Case:** Entry: 10, Exit: 14, Vehicle: 1 (Car), Overnight: 0.
* **Expected Output:**

```text
Parking Duration: 4 hours
Parking Fee: Rs.120
```

Program 19: Multi-Product Shopping Budget Constraint

* **Problem Statement:** Manage shopping cart purchases for multiple products against a predefined budget, automatically trimming quantities if the budget is exceeded.
* **Objectives:** Compute total shopping cost, compare against budget, and sequentially reduce item quantities from the last product until the cost fits within the budget.
* **Assumptions:** Maximum 50 products, prices and quantities are non-negative.

Algorithm & Pseudocode

1. Input number of products ($N \le 50$) and budget.
2. Loop to input price and quantity for each product into arrays.
3. Compute total cost = $\sum (\text{price}[i] \times \text{quantity}[i])$.
4. If $\text{total} \le \text{budget}$, shopping is within budget.
5. Else, iterate backwards from product $N-1$ to 0, decrementing quantity unit by unit until $\text{total} \le \text{budget}$. Display revised list.

Python Source Code

```python
def shopping_budget():
    n = int(input("Enter number of products (max 50): "))
    if not (1 <= n <= 50):
        print("Invalid number of products!")
        return
```

```
budget = int(input("Enter budget: "))
if budget < 0:
    print("Invalid budget!")
    return

prices = []
quantities = []
for i in range(n):
    p = int(input(f"Product {i+1} price: "))
    q = int(input("Quantity: "))
    if p < 0 or q < 0:
        print("Invalid product data!")
        return
    prices.append(p)
    quantities.append(q)

total = sum(p * q for p, q in zip(prices, quantities))
print(f"\nTotal Shopping Cost: Rs.{total}")

if total <= budget:
    print("Shopping is within budget.")
else:
    print(f"Budget exceeded by Rs.{total - budget}")
    for i in range(n - 1, -1, -1):
        while quantities[i] > 0 and total > budget:
            quantities[i] -= 1
            total -= prices[i]

    print("\nRevised Shopping List:")
```

```
        for i in range(n):
            print(f"Product {i+1}: {quantities[i]} units")
        print(f"Final Cost: Rs.{total}")

if __name__ == "__main__":
    shopping_budget()

```

* **Test Case:** 2 products. Product 1: Price 100, Qty 3. Product 2: Price 200, Qty 2. Budget = 500.
* **Expected Output:**

```text
Total Shopping Cost: Rs.700
Budget exceeded by Rs.200
Revised Shopping List:
Product 1: 3 units
Product 2: 1 units
Final Cost: Rs.500

```

Program 20: University Admission Decision Engine

* **Problem Statement:** Evaluate candidate eligibility based on aggregate marks and subject clearance, then sort and rank candidates by eligibility status and marks.

* **Objectives:** Determine eligibility (marks ≥ 60 and subject flag == 1), sort candidates in descending order of eligibility and marks, and output the ranked admission list.
* **Assumptions:** Maximum 50 candidates, marks range from 0 to 100.

Algorithm & Pseudocode

1. Input number of candidates ($N \le 50$).
2. For each candidate, input aggregate marks and subject eligibility flag.
3. Determine eligibility: `eligible = 1` if subject == 1 and marks ≥ 60, else `0`.
4. Sort candidates using bubble sort / custom key: primary key `eligible` (descending), secondary key `marks` (descending).
5. Print final admission ranking.

Python Source Code

```python
def admission_engine():
    n = int(input("Enter number of candidates (max 50): "))
    if not (1 <= n <= 50):
        print("Invalid number of candidates!")
        return

    candidates = []
    for i in range(n):
        print(f"\nCandidate {i+1}")
        marks = int(input("Enter aggregate marks (0-100): "))
        subject = int(input("Subject eligibility (1=Yes, 0=No): "))

        if not (0 <= marks <= 100 and subject in [0, 1]):
```

```
            print("Invalid input!")
            return

        eligible = 1 if (subject == 1 and marks >= 60) else 0
        candidates.append({'id': i+1, 'marks': marks, 'subject': subject, 'eligible': eligible})

    candidates.sort(key=lambda x: (-x['eligible'], -x['marks']))

    print("\nAdmission Ranking:")
    for rank, c in enumerate(candidates, 1):
        status = "Eligible" if c['eligible'] else "Not Eligible"
        print(f"Rank {rank}: Marks = {c['marks']}, {status}")

if __name__ == "__main__":
    admission_engine()
```

* **Test Case:** 2 candidates. Candidate 1: Marks 85, Subject 1. Candidate 2: Marks 90, Subject 0.
* **Expected Output:**

```text
Admission Ranking:
Rank 1: Marks = 85, Eligible
Rank 2: Marks = 90, Not Eligible
```

---

3. Quick Summary of Programs 12, 14, 16, 17, and 18

Program	Core Logic / Purpose	Key Inputs	Output Metrics
12. Attendance Recovery	Computes future attendance percentage and consecutive classes needed to meet target.	Attended, Total, Future classes, Required %	Projected % & Min consecutive classes required
14. Warehouse Inventory	Reconciles stock discrepancies and checks replenishment thresholds (< 10 units).	Opening, Incoming, Dispatched, Actual Closing	Expected vs Actual difference, Replenishment alert
16. Water Tank Simulation	Simulates multi-interval water inflow/outflow with overflow/shortage detection.	Capacity, Initial water, Threshold, Intervals	Water level per interval, overflow/shortage alerts
17. Performance Trend	Compares previous vs. current exam marks across subjects to classify student performance.	Previous marks, Current marks per subject	Improvement/Decline delta, Overall classification
18. Fuel Cost Comparison	Calculates journey fuel requirements and total cost across multiple vehicles to find the cheapest option.	Distance, Vehicle mileage, Fuel price, Reserve	Fuel liters, Cost per vehicle, Optimal vehicle ID

---

4. Conclusion

The structured analysis, algorithms, and Python equivalents provided above translate procedural C logic into clean, readable, and robust Python scripts. Each program successfully fulfills its designated domain requirement—whether validating grades,

processing financial transactions, managing inventories, or optimizing route costs—ensuring error-free execution and clear user feedback.